

Cursive writing app idea



I want to create an iPad app that uses handwriting recognition to teach cursive writing. It'll teach students to write English cursive by showing them example text and having them copy it using the Apple Pencil. The app will use the iPad's 'scribble' feature to turn handwriting into editable text. The built-in text recognition feature will evaluate the handwriting and identify areas for improvement.



Got it — you're essentially envisioning a "learn-to-type" app, but instead of a keyboard, the Apple Pencil + iPad's handwriting recognition becomes the input method, with live feedback on cursive accuracy.

Here's how this could work in practice, and what you'd need to account for:

Core Flow

1. Lesson Display

- Show a target sentence or letter in cursive (animated stroke-by-stroke to model correct formation).
- Include visual cues for start/stop points and stroke direction.

2. Student Writing

- Student uses the Apple Pencil to write directly into a text field or dedicated drawing canvas.
- While writing, the *Scribble* system converts strokes to text in real time.

3. Text Recognition Feedback

- Once written, compare the recognized text to the target text.
- Use:
 - **Spelling match** → to check letter recognition & accuracy.
 - **Timing & stroke analysis** (if captured) → to assess speed and flow.

4. Error Detection

- If Scribble misreads a letter, the mismatch can indicate:
 - Letter shape/form issues
 - Incorrect connections
 - Extra or missing strokes
- Visual highlight of problem letters (color overlay or underline).

5. Coaching Feedback

- Provide specific hints:
 - “Loop in your lowercase ‘e’ is too tight.”
 - “Tall letters like ‘l’ should rise above midline.”
- Replay an animation showing correct form.

Tech Considerations

1. Scribble API Access

- **Challenge:** Apple doesn't expose the full raw handwriting stroke data from Scribble to developers — you typically get only the converted text.
- **Solution:**
 - Use ``PencilKit`` to collect raw strokes yourself.
 - Run them through ``Vision`` framework's handwriting recognition for more control.
 - Optionally, use Core ML custom models for style/scoring analysis.

2. Recognizing Cursive

- Apple's built-in handwriting recognizer is mostly designed for legibility, not stylistic scoring.
- You'll likely need:
 - Your own ML model trained on cursive handwriting samples.
 - Rules for letter height, slant, and joins.
 - Tolerance thresholds for different student levels.

3. UI/UX

- Cursive-friendly writing area (lines, baselines, midlines like notebook paper).
- Real-time stroke smoothing and gentle animation cues.
- Progress tracking dashboard for each student.

4. Gamification

- Levels that unlock as accuracy improves.
 - Timed challenges (“Write this sentence in 15 seconds!”).
 - Rewards for consistent slant and clean joins.
-

Rough Tech Stack

- **Frontend:** SwiftUI or UIKit (iPad-optimized)
 - **Drawing Input:** PencilKit
 - **Handwriting Recognition:** Vision + Core ML (custom model for cursive grading)
 - **Text Comparison:** String matching, Levenshtein distance for partial matches
 - **Feedback Logic:** Custom scoring engine (accuracy %, legibility rating)
 - **Data Storage:** Core Data or CloudKit for syncing progress
-

If you want, I can break this down into a **full development blueprint** with the exact Swift code architecture, including:

- How to capture and process PencilKit strokes
- How to integrate Vision handwriting recognition
- How to score and give feedback on cursive quality

That would get you from *concept* to *prototype-ready*. Would you like me to map that out next?



That would be great!



Alright — let's map this out so you can see the full pipeline from **Apple Pencil stroke** → **recognition** → **feedback** for a cursive tutor app.

I'll break it into **architecture**, **data flow**, and **prototype-level implementation steps**.

1. App Architecture Overview

```
[Lesson Engine] → [PencilKit Canvas] → [Stroke Data]
  → (A) Vision Recognition (text)
  → (B) Cursive Scoring Engine (style)
  → [Feedback UI] → [Progress Tracking]
```

Key components:

1. Lesson Engine

- Displays target letters/words in cursive (animated stroke-by-stroke).
- Provides line guides (baseline, x-height, ascender/descender lines).

2. PencilKit Canvas

- Captures *raw strokes* (not just the converted text).
- Allows full analysis of stroke position, direction, speed.

3. Recognition Pipeline

- **Vision Framework:** Converts strokes → recognized text.
- **Custom Analysis:** Compares stroke geometry to a reference model of cursive.

4. Cursive Scoring Engine

- Checks:
 - Letter formation (shape match to ideal template)
 - Slant angle
 - Connection smoothness
 - Relative heights
- Returns a per-letter accuracy score.

5. Feedback UI

- Highlights incorrect letters (red/amber/green).
- Offers replay animation for correct form.
- Gives a text note (“Your ‘g’ descender is too short”).

6. Progress Tracking

- Stores lesson completion, scores, time spent.
- Uses Core Data / CloudKit for persistence & sync.

2. Data Flow

Example for writing the word *loop*:

1. Show Target

- Animated cursive “loop”
- Guidelines visible

2. User Writes

- PencilKit records strokes in `PKDrawing``
- Stores `PKStrokePoint`` data: (x, y, pressure, azimuth, timestamp)

3. Process Strokes

- Send to Vision (`VNRecognizeTextRequest``) to get recognized text
- Parallel: Run stroke comparison algorithm

4. Evaluate

- Text match?  / 
- Shape match score: 0-100
- Identify weak points (e.g., slant too steep)

5. Display Feedback

- Overlay corrections visually
- Provide short actionable note

6. Save

- Update user’s performance history

3. Prototype Implementation Steps

Step 1 - Set Up PencilKit Canvas

```
import SwiftUI
import PencilKit

struct WritingView: UIViewRepresentable {
    @Binding var canvasView: PKCanvasView

    func makeUIView(context: Context) -> PKCanvasView {
        canvasView.drawingPolicy = .anyInput
        canvasView.backgroundColor = .white
        canvasView.allowsFingerDrawing = false
        return canvasView
    }

    func updateUIView(_ uiView: PKCanvasView, context: Context) {}
}
```

- Use lined paper background via a `CALayer`.
- Keep `PKCanvasView` persistent between lessons to store strokes.

Step 2 - Recognize Text (Vision Framework)

```
import Vision

func recognizeHandwriting(from image: UIImage, completion: @escaping (String) -> Void) {
    let request = VNRecognizeTextRequest { request, _ in
        let recognized = request.results?
            .compactMap { ($0 as? VNRecognizedTextObservation)?.topCandidates(1).first?.string }
            .joined(separator: " ") ?? ""
        completion(recognized)
    }
    request.recognitionLanguages = ["en-US"]
}
```



```

request.recognitionLevel = .accurate
request.usesLanguageCorrection = true

let handler = VNImageRequestHandler(cgImage: image.cgImage!)
try? handler.perform([request])
}

```

- Convert `PKDrawing` → `UIImage` and pass to this function.
- This gives you what Vision *thinks* the student wrote.

Step 3 - Compare Against Target

```

func compareText(_ target: String, _ student: String) -> Double {
    // Simple similarity score using Levenshtein distance
    let distance = levenshtein(target.lowercased(), student.lowercased())
    let maxLen = max(target.count, student.count)
    return 1.0 - (Double(distance) / Double(maxLen))
}

```

- Combine with per-letter bounding box comparison from stroke data for better precision.

Step 4 - Cursive Stroke Scoring

- Extract:
 - **Slant angle:** Fit a regression line to stroke points; measure deviation from target slant (e.g., 75° from horizontal).
 - **Proportions:** Compare letter heights relative to baseline.
 - **Join smoothness:** Measure curve continuity at letter joins.

- This requires geometric math on the `(x, y)` stroke data.
-

Step 5 - Feedback Rendering

- Overlay student's strokes in semi-transparent red on top of correct template in green.
 - Show "ghost" animation of correct motion.
 - Give short text advice.
-

Step 6 - Progress Tracking

- Store results in Core Data:

```
LessonResult: {  
  lessonID: String,  
  date: Date,  
  accuracyScore: Double,  
  avgSlantError: Double,  
  notes: String  
}
```

- Use CloudKit for multi-device sync.
-

4. Next-Level Features

- **Adaptive Difficulty:** AI picks the next lesson based on recent struggles.

- **Gamification:** XP, streaks, unlockable “calligraphy styles.”
 - **Offline Mode:** Pre-trained Core ML cursive evaluator for speed.
-

If you want, I can also draft a **stroke analysis algorithm** that measures slant, letter size, and joins from PencilKit data — that’s the missing piece that makes this *really* a cursive evaluator rather than just a handwriting recognizer.

That’s the part Apple doesn’t give you out-of-the-box, so it’s where your app can stand out.

Do you want me to go ahead and design that algorithm?



“ ... make it so. ” (yes please. :).)



Heck yes — let’s make it so. 😊

Below is a practical, prototype-ready **stroke analysis algorithm** you can drop into your iPad app. I’ll give you:

1. high-level approach
2. feature extraction details (math + intuition)
3. per-letter segmentation strategy
4. scoring rubric (how to combine features into useful feedback)
5. Swift code snippets (PencilKit → features → scores)

6. visualization / UX notes

If you want, I can convert any piece into a full working Swift file next.

1) High-level approach (pipeline)

1. Capture raw strokes from ``PKDrawing`` → sequence of ``PKStrokePoint``s (x,y,time,pressure,azimuth/altitude).
2. Preprocess: normalize for scale and baseline, smooth noisy points, resample.
3. Segment strokes into graphemes/letters using spatial-temporal heuristics and Vision text bounding boxes.
4. For each letter (or multi-letter join) extract features:
 - Slant angle
 - Height ratios (x-height, ascender, descender)
 - Stroke direction & ordering
 - Curvature / smoothness at joins
 - Connection integrity between letters
 - Timing & pressure patterns
5. Compare features to canonical templates (either rule-based templates or a small ML model).
6. Produce per-letter scores + human-readable hints.
7. Combine with Vision text recognition (did Scribble read it as the intended letter?) to prioritize remediation.

2) Preprocessing (concrete steps)

- **Collect points:** for each ``PKStroke``, read ``PKStrokePath`` → ``PKStrokePoint`` (x,y,time,force).
- **Smoothing:** use a low-pass filter (moving average or a 3-point Savitzky-Golay) to remove jitter.
- **Resample:** re-sample strokes at a fixed arc-length spacing (e.g., every 2–3 px) so comparisons are stable.
- **Scale & Align:**
 - Detect baseline and mean line (see baseline detection below).
 - Translate so baseline = 0, scale horizontally/vertically to match template metrics (or keep absolute but compute ratios).
- **Optional:** rotate to canonical slant for shape-only comparisons (but keep raw slant for slant scoring).

3) Baseline detection & line metrics

Compute baseline (most important):

- Collect all ``y`` coordinates from strokes in the writing area.
- Build a histogram of y positions (or run RANSAC line fit on stroke endpoints). The baseline will be the dominant horizontal mode where most letter bottoms sit.
- Compute ``xHeight`` (median height of most frequent stroke midpoints) and ``ascenderLine``/``descenderLine`` from percentiles (e.g., 85th / 15th percentiles relative to baseline).

These give you relative metrics (ascender height in px, x-height in px) for proportion checks.

4) Segmentation into letters (practical heuristics)

You'll want per-letter scores. Two approaches used together:

A) **Vision bounding boxes**: Render `PKDrawing` to an image, run `VNRecognizeTextRequest` with `.accurate`. Use `VNRecognizedTextObservation` bounding boxes to get rough letter/word bounds.

B) **Spatial-temporal clustering**: For strokes not neatly separated:

- Compute stroke centroids $(\bar{x}, \bar{y})$ and stroke end times.
- Cluster strokes into letters by proximity (distance threshold relative to x-height) and time gap (pause $> 250\text{--}500$ ms = separator).

Combine A + B: use Vision boxes to anchor segmentation; fill gaps with clustering when Vision is ambiguous.

5) Feature extraction (per letter)

a) Slant angle

- For all points of the letter, do PCA (principal component analysis) or linear regression on (x, y) to get main axis.
- Slant = $\arctan2(\text{PC1}_y, \text{PC1}_x)$ in degrees relative to vertical/horizontal convention you choose.

- ``slantError = abs(slant - targetSlant)`` where ``targetSlant`` might be $\sim -15^\circ$ to -30° (depends on style). Use signed error for hints (“too upright”, “too slanted”).

b) Height / proportion metrics

- Compute bounding box heights:
 - ``totalHeight = maxY - minY``
 - ``midline`` approximate = median y of refined stroke segments
- Ratios:
 - ``ascenderRatio = (ascenderLine - baseline)/xHeight``
 - ``xHeightRatio = xHeight / totalHeight``
- Compare to canonical ratios. Flag if ``xHeightRatio`` too small/large.

c) Curvature & smoothness

- Compute curvature κ along the stroke: for parametric points $p(t)$, curvature $\approx |x' y'' - y' x''| / ((x'^2 + y'^2)^{3/2})$.
- Summarize: mean curvature, variance of curvature, max curvature at joins.
- Smooth joins have low curvature variance and small spikes. Large spikes at expected loops are okay; spikes at joins indicate jaggedness.

d) Connection integrity

- For letters that should connect (e.g., “l”→“o”), check distance between the end point of one letter and start point of next relative to x-height. A small gap or overlap indicates an incorrect join.
- Also check tangent direction: the incoming and outgoing tangent vectors should be roughly aligned.

e) Stroke ordering & direction (optional but powerful)

- If you have stroke order expectations, check whether the major strokes follow typical order (start at top for many letters, etc.). Differences don't always mean incorrect penmanship, but can identify systematic problems for younger learners.

f) Timing & pressure

- Compute average stroke speed (arc-length / time) and pressure force statistics. Extremely slow/fast or wildly varying pressure can imply poor fluency.

g) Shape similarity to template

- After normalizing scale & rotation (but keep slant separate), compute a shape distance between the student stroke polyline and canonical letter template using Dynamic Time Warping (DTW) on x,y sequences or Hausdorff distance.

6) Scoring rubric (combine features)

Produce a per-letter score (0-100). Example weighted combination:

- Legibility match (Vision/Levenshtein / text match): 30%
 - If Vision reads intended letter → high partial credit.
- Shape similarity (DTW/Hausdorff): 25%
- Slant correctness: 15%
- Height/proportion: 10%
- Join smoothness/connection: 10%
- Timing/pressure fluency: 5%



- Stroke order/direction: 5%

Normalize each subscore to 0–100, then `finalScore` = weighted sum.

Thresholds for feedback:

- 90–100: Great — minor polish
- 75–89: Good — targeted hints
- 50–74: Needs work — show ghost animation + practice
- <50: Redo letter drills

Also provide *explainable hints* by checking which individual feature is worst: e.g., if `slantError` is largest contributor → hint about slant and show correction animation.

7) Example Swift snippets

a) Extract points from `PKDrawing`

```
import PencilKit

func extractPoints(from drawing: PKDrawing) -> [[PKStrokePoint]] {
    var strokesPoints: [[PKStrokePoint]] = []
    for stroke in drawing.strokes {
        var pts: [PKStrokePoint] = []
        stroke.path.forEach { element in
            // PKStrokePath provides an API to iterate—pseudocode here
            // You can sample using stroke.path.interpolatedPoints(...) or
            stroke.path.points
        }
        strokesPoints.append(pts)
    }
    return strokesPoints
}
```

"Note: `PKStrokePath` methods are not public for direct point iteration in older SDKs; use `stroke.path` sampling functions (e.g., `forEach` on `PKStrokePath`), or convert to `UIImage` and extract contours in Vision if needed. Newer SDKs expose `PKStrokePoint`."

b) Simple slant via linear regression (points -> slope)

```
import CoreGraphics

func slantAngleDegrees(points: [CGPoint]) -> Double {
    guard points.count > 2 else { return 0 }
    let xs = points.map { Double($0.x) }
    let ys = points.map { Double($0.y) }
    let xMean = xs.reduce(0,+) / Double(xs.count)
    let yMean = ys.reduce(0,+) / Double(ys.count)
    let num = zip(xs, ys).map { ($0 - xMean)*($1 - yMean) }.reduce(0,+)
    let den = xs.map { ($0 - xMean)*($0 - xMean) }.reduce(0,+)
    if den == 0 { return 0 }
    let slope = num / den // dy/dx
    let angleRad = atan(slope)
    return angleRad * 180.0 / .pi
}
```

(Interpretation note: depending on your coordinate system, sign/direction may invert.)

c) Curvature (discrete)

```
func curvature(points: [CGPoint]) -> [Double] {
    guard points.count >= 3 else { return [] }
    var k: [Double] = []
    for i in 1..
```

```

        // simpler stable approximation:
        let denom2 = pow((dx1*dx1 + dy1*dy1), 1.5) + 1e-9
        let curvatureApprox = cross / denom2
        k.append(curvatureApprox)
    }
    return k
}

```

(Use a robust curvature formula or fit local circles for better accuracy.)

d) DTW shape distance (very small sketch)

You can either implement DTW in Swift or compute normalized point distances. DTW pseudocode: compute matrix of distances and dynamic program minimal warping cost. (If you want, I'll produce the full DTW Swift implementation.)

8) Example per-letter feedback logic (pseudocode)

```

for each letter:
    compute features
    compute individual scores:
        s_legibility = visionMatch? 100 : (80 * textSimilarity)
        s_shape = 100 - 100 * normalizedShapeDist
        s_slant = max(0, 100 - (slantErrorDeg * slantWeight))
        ...
    finalScore = weightedSum(...)
    if finalScore < 75:
        find largest deficit feature -> produce hint

```

Example hint selection:

- largest deficit is `slant` → "Try angling your letters slightly to the right; practice with slanted guide lines."

- largest deficit is `joinSmoothness` → "Work on connecting strokes: don't lift at joins and keep motion fluid."
-

9) Templates & training data

You'll need canonical templates. Options:

- Draw canonical cursive letters yourself (vector templates).
- Use averaged strokes from sample users / teachers to make multiple style templates.
- Or train a small Core ML model (classification/regression) with labeled examples for more robust scoring.

For quick prototyping, rule-based templates + DTW to a teacher-drawn template are excellent.

10) UX & visualization tips

- Show a "ghost" stroke animation of the ideal letter overlaid as the student practices.
 - Highlight the letter areas causing trouble (red overlay). Let student tap a highlighted letter to see the detailed breakdown (slant, height, curvature).
 - Provide drills: isolate the worst feature (e.g., slant practice line, loop practice).
 - Keep feedback bite-sized: one or two actionable tips per attempt.
-

11) Performance considerations

- Do heavy geometric calculations off the main thread.
 - Cache computed features for repeated small updates (e.g., when student writes multiple times).
 - If using Core ML, quantize and run on-device for offline real-time feedback.
-

Want the next step?

I can:

- produce a **full Swift file** that:
 - extracts stroke points from ``PKDrawing``,
 - resamples/smooths them,
 - computes slant, height ratios, curvature,
 - does DTW shape distance,
 - outputs a JSON per-letter report,
— or —
- produce **DTW & curvature** unit-tested implementations in Swift,
- or design a small **training dataset schema** and a Core ML model spec for a cursive evaluator.

Which of those would you like first? (I recommend the full Swift prototype file so you can drop it into a sample app.)



yes i agree with the full Swift prototype as next step. Also i am very excited about this project and the level of detail in the process and precision in each piece and its technical requirements and the explanation for the specific design and execution of this. as well as the pieces themselves and their development into a whole. that consistently helps me understand and demonstrates your understanding of the nature of the intended overall product.



Thought for 4 seconds

- Thanks for your excitement! I'm thrilled to share the full Swift prototype you're looking for. This version will include everything you asked for, from extracting stroke points to computing slant, curvature, and more. I'll keep it smooth and efficient, using PencilKit and background threads for optimal performance. Here's the full code in a single Swift file. Enjoy!

✓ Done

Love the energy — let's build the prototype. Below is a single Swift file you can drop into an Xcode iPad project (iOS 15+ recommended).

It captures ``PKDrawing`` strokes, extracts & preprocesses points, computes slant / height ratios / curvature, runs a DTW shape distance against a teacher template, segments letters using Vision bounding boxes as anchors, and finally emits a per-letter JSON report (with scores and feature breakdowns).

Notes before you paste/run:

- This is **prototype** code meant to be clear and practical; you'll want to adapt thresholds, canonical templates, and edge-case handling to your app.
- Some ``PKStrokePath`` internals have changed across SDKs; I included a small fallback that rasterizes the drawing to an image if direct point sampling isn't available in your SDK.

- Heavy math runs off the main thread. Hook the API into your UI pipeline (e.g., call ``CursiveAnalyzer.analyze(drawing: target:)`` when user completes a word).
- You'll need to provide teacher templates (vectors) for DTW comparisons — I included a sample letter template format and helper to register templates.

Full Swift prototype file — ``CursiveAnalyzer.swift``

```
//  
//  CursiveAnalyzer.swift  
//  CursiveTutorPrototype  
//  
//  Created by You on YYYY-MM-DD.  
//  
  
import Foundation  
import PencilKit  
import Vision  
import CoreGraphics  
import UIKit  
  
// MARK: - Public API  
  
/// Result objects that will be JSON-serializable  
struct LetterReport: Codable {  
    let letter: String  
    let boundingBox: CGRect // normalized to [0,1] within the drawing image  
    let finalScore: Double // 0..100  
    let featureScores: FeatureScores  
    let notes: [String]  
}  
  
struct FeatureScores: Codable {  
    let legibilityScore: Double
```

```

    let shapeSimilarityScore: Double
    let slantScore: Double
    let proportionScore: Double
    let connectionScore: Double
    let curvatureScore: Double
    let timingScore: Double
}

/// Top-level analysis result
struct AnalysisReport: Codable {
    let timestamp: Date
    let overallScore: Double
    let letters: [LetterReport]
}

/// Main analyzer entrypoint
final class CursiveAnalyzer {
    static let shared = CursiveAnalyzer()
    private init() {}

    // Teacher templates: map letter -> array of CGPoint (normalized)
    // You must register templates before using DTW comparisons.
    private var templates: [String: [CGPoint]] = [:]

    // Register a canonical stroke template (normalized between 0-1)
    func registerTemplate(letter: String, normalizedPoints: [CGPoint]) {
        templates[letter.lowercased()] = normalizedPoints
    }

    /// Main analysis function.
    /// - Parameters:
    ///   - drawing: PKDrawing from the user's canvas
    ///   - targetText: intended text string (e.g., "loop")
    ///   - completion: returns AnalysisReport or error
    func analyze(drawing: PKDrawing, targetText: String, completion: @escaping
(Result<AnalysisReport, Error>) -> Void) {
        DispatchQueue.global(qos: .userInitiated).async {
            do {
                // 1. extract strokes -> stroke groups of points
                let strokes = try StrokeExtractor.extractStrokePoints(from:
drawing)

```



```

        // 2. rasterize to image for Vision bounding boxes (and to
        normalize coordinates)
        let image = drawing.asImage(backgroundColor: .white, scale:
3.0)

        // 3. run VNRecognizeTextRequest to get letter bounding boxes
        (word/letter anchors)
        let observations = try
VisionHelpers.recognizeTextObservations(in: image)

        // 4. segment strokes into letters using boxes + clustering
        heuristics
        let letterSegments = Segmenter.segment(strokes: strokes,
observations: observations, imageSize: image.size)

        // 5. compute baseline and line metrics for the whole drawing
        let baselineInfo = BaselineDetector.detectBaseline(from:
strokes)

        // 6. for each letter segment -> compute features & scores
        var letterReports: [LetterReport] = []
        var scoreSum: Double = 0
        for seg in letterSegments {
            // get intended letter mapping from recognition (best
            guess) or targetText fallback
            let intendedLetter = seg.recognizedString?.lowercased() ??
"?"

            let pts = Preprocessor.resampleAndSmooth(points:
seg.points, spacing: 2.5)
            let normalizedPts = CoordinateNormalizer.normalize(points:
pts, in: CGRect(origin: .zero, size: image.size))

            // features
            let slantDeg = FeatureExtractor.slantAngleDegrees(points:
normalizedPts)
            let curvatureVals = FeatureExtractor.curvatures(points:
normalizedPts)
            let curvatureSummary = curvatureVals.isEmpty ? 0.0 :
curvatureVals.mean()
            let bbox = seg.boundingBox ?? CGRect.zero

```

```

        let proportions = FeatureExtractor.proportions(points:
normalizedPts, baselineInfo: baselineInfo, drawingSize: image.size)

        // shape similarity (DTW) vs template if exists
        let dtwScore: Double
        if let template = self.templates[intendedLetter] {
            // templates assumed normalized to 0..1 already
            let dist = DTW.distance(sequenceA: normalizedPts,
sequenceB: template)

            // convert distance to similarity score heuristically
            dtwScore = max(0, 100 - (dist * 1000)) // adjust scale
based on your templates
        } else {
            dtwScore = 50 // neutral when no template
        }

        // legibility via Vision: compare recognized char to
intended char

        let legibility: Double = {
            guard let rec = seg.recognizedString?.lowercased() else
{ return 0.0 }

            return rec == intendedLetter ? 100.0 :
(StringSimilarity.levenshteinSimilarity(a: intendedLetter, b: rec) * 100.0)
        }()

        // individual sub-scores (0..100)
        let slantScore = Scoring.slantScore(for: slantDeg,
targetSlantDeg: -20, toleranceDeg: 10) // target -20°, ± tolerance
        let proportionScore = Scoring.proportionScore(xHeightRatio:
proportions.xHeightRatio, ideal: 0.45, tolerance: 0.18)
        let curvatureScore = Scoring.curvatureScore(meanCurvature:
curvatureSummary)

        let connectionScore = Scoring.connectionScore(for: seg,
baselineInfo: baselineInfo)
        let timingScore = Scoring.timingScore(for: seg)

        // compose full score with weights
        let weights = Scoring.Weights()
        let finalScore = weights.legibility * (legibility) +
                        weights.shape * dtwScore +
                        weights.slant * slantScore +

```

```

        weights.proportion * proportionScore +
        weights.connection * connectionScore +
        weights.curvature * curvatureScore +
        weights.timing * timingScore

let featureScores = FeatureScores(
    legibilityScore: legibility,
    shapeSimilarityScore: dtwScore,
    slantScore: slantScore,
    proportionScore: proportionScore,
    connectionScore: connectionScore,
    curvatureScore: curvatureScore,
    timingScore: timingScore
)

// produce textual hints
var notes: [String] = []
if finalScore < 75 {
    // find worst contributor
    let map: [(String, Double)] = [
        ("legibility", legibility),
        ("shape", dtwScore),
        ("slant", slantScore),
        ("proportion", proportionScore),
        ("connection", connectionScore),
        ("curvature", curvatureScore),
        ("timing", timingScore)
    ]
    if let worst = map.min(by: { $0.1 < $1.1 }) {
        switch worst.0 {
        case "slant":
            notes.append("Try angling your letters slightly
more to the right; use slanted guide-lines.")
        case "proportion":
            notes.append("Work on letter height: midline
should be about half of full height.")
        case "connection":
            notes.append("Practice smooth joins between
letters; avoid lifting at the connection.")
        case "curvature":
            notes.append("Loops are jagged – aim for

```

```

smoother circular motions.")
        case "legibility":
            notes.append("Letter was misrecognized – focus
on clear letter shape.")
        case "shape":
            notes.append("Shape differs from the model –
follow the ghost animation carefully.")
        case "timing":
            notes.append("Try to keep a consistent, smooth
writing speed.")
        default:
            notes.append("Keep practicing this letter with
the guided drill.")
    }
}
} else {
    notes.append("Nice! Minor polish recommended.")
}

let letterReport = LetterReport(
    letter: intendedLetter,
    boundingBox: bbox,
    finalScore: min(max(finalScore, 0.0), 100.0),
    featureScores: featureScores,
    notes: notes
)

letterReports.append(letterReport)
scoreSum += letterReport.finalScore
}

let overall = letterReports.isEmpty ? 0.0 : scoreSum /
Double(letterReports.count)
let report = AnalysisReport(timestamp: Date(), overallScore:
overall, letters: letterReports)
DispatchQueue.main.async {
    completion(.success(report))
}
} catch {
    DispatchQueue.main.async {
        completion(.failure(error))
    }
}

```

```

    }
    }
}

// MARK: - Stroke extraction

enum StrokeExtractionError: Error {
    case cannotExtractPoints
}

fileprivate struct StrokeExtractor {
    /// Returns array of strokes; each stroke is array of CGPoints in original
    drawing space
    static func extractStrokePoints(from drawing: PKDrawing) throws ->
    [[CGPoint]] {
        // try direct extraction from PKDrawing.strokes
        var all: [[CGPoint]] = []
        if !drawing.strokes.isEmpty {
            for stroke in drawing.strokes {
                // PKStroke provides `path` which can be sampled.
                // We'll try `forEach` to append points; if not available,
                fallback to raster approach below.
                var pts: [CGPoint] = []
                stroke.path.forEach { element in
                    switch element {
                    case .moveToPoint(let p):
                        pts.append(CGPoint(x: CGFloat(p.x), y: CGFloat(p.y)))
                    case .addLineToPoint(let p):
                        pts.append(CGPoint(x: CGFloat(p.x), y: CGFloat(p.y)))
                    case .addQuadCurveToPoint(let p, control: let c):
                        // approximate quadratic by adding control then end
                        pts.append(CGPoint(x: CGFloat(c.x), y: CGFloat(c.y)))
                        pts.append(CGPoint(x: CGFloat(p.x), y: CGFloat(p.y)))
                    case .addCurveToPoint(let p, control1: let c1, control2:
let c2):
                        pts.append(CGPoint(x: CGFloat(c1.x), y: CGFloat(c1.y)))
                        pts.append(CGPoint(x: CGFloat(c2.x), y: CGFloat(c2.y)))
                        pts.append(CGPoint(x: CGFloat(p.x), y: CGFloat(p.y)))
                    @unknown default:

```

```

        break
    }
}
if !pts.isEmpty {
    all.append(pts)
}
}
if !all.isEmpty { return all }
}

// Fallback: rasterize the drawing and extract contours via Vision's
contours request
// (not as precise but works if direct sampling is unavailable)
guard let image = drawing.asImage(backgroundColor: .white, scale:
3.0).cgImage else {
    throw StrokeExtractionError.cannotExtractPoints
}
let contours = try VisionHelpers.extractContours(from: image)
return contours
}
}

// MARK: - Preprocessing utilities

fileprivate struct Preprocessor {
    /// Moving-average smoothing then resample to uniform spacing (arc-length)
    static func resampleAndSmooth(points: [CGPoint], spacing: CGFloat) ->
[CGPoint] {
        guard points.count > 2 else { return points }
        // smoothing: simple moving average of window 3
        var smoothed: [CGPoint] = []
        for i in 0..

```

```

    for i in 1..

```

// MARK: - Baseline detection & proportions

```

fileprivate struct BaselineInfo {
    let baselineY: CGFloat // y in drawing coordinates
    let xHeight: CGFloat
    let ascenderLine: CGFloat
    let descenderLine: CGFloat
}

fileprivate struct BaselineDetector {

```

```

static func detectBaseline(from strokes: [[CGPoint]]) -> BaselineInfo {
    // gather all y-values of stroke points
    var ys: [CGFloat] = []
    for stroke in strokes {
        for p in stroke { ys.append(p.y) }
    }
    if ys.isEmpty {
        return BaselineInfo(baselineY: 0, xHeight: 10, ascenderLine: 20,
descenderLine: -10)
    }
    // histogram approach: bucket y values
    let sorted = ys.sorted()
    let median = sorted[sorted.count/2]
    // xHeight: use interquartile range to find typical stroke midline size
    let q1 = sorted[max(0, sorted.count/4)]
    let q3 = sorted[min(sorted.count-1, sorted.count*3/4)]
    let iqr = q3 - q1
    // baseline is near the median of lower half (letters sit above
baseline)
    let baseline = q3 // heuristic
    let xHeight = max(6.0, iqr * 0.6)
    let asc = baseline - (xHeight * 0.9)
    let desc = baseline + (xHeight * 1.2)
    return BaselineInfo(baselineY: baseline, xHeight: xHeight,
ascenderLine: asc, descenderLine: desc)
}

// MARK: - Segmenter (Vision + clustering heuristics)

fileprivate struct Segment {
    let points: [CGPoint]
    let boundingBox: CGRect? // in image coordinates
    let recognizedString: String?
}

fileprivate struct Segmenter {
    static func segment(strokes: [[CGPoint]], observations:
[VNRecognizedTextObservation], imageSize: CGSize) -> [Segment] {
        // Build segments anchored by Vision boxes (word-level or character-
level)

```



```

// Map each stroke to the box it intersects most strongly
var strokeAssigned = Array(repeating: false, count: strokes.count)
var boxToStrokes: [(obs: VNRecognizedTextObservation, strokesIdx:
[Int])] = []
for obs in observations {
    var assigned: [Int] = []
    let bbox = VNImageRectForNormalizedRect(obs.boundingBox,
Int(imageSize.width), Int(imageSize.height))
    for (i, stroke) in strokes.enumerated() {
        // compute stroke bounding box
        let sMinX = stroke.map { $0.x }.min() ?? 0
        let sMaxX = stroke.map { $0.x }.max() ?? 0
        let sMinY = stroke.map { $0.y }.min() ?? 0
        let sMaxY = stroke.map { $0.y }.max() ?? 0
        let sbbox = CGRect(x: sMinX, y: sMinY, width: sMaxX - sMinX,
height: sMaxY - sMinY)
        if sbbox.intersects(bbox) {
            assigned.append(i)
            strokeAssigned[i] = true
        }
    }
    boxToStrokes.append((obs, assigned))
}

// leftover strokes -> cluster by x-proximity into their own segments
var leftoverIndices: [Int] = []
for (i, used) in strokeAssigned.enumerated() {
    if !used { leftoverIndices.append(i) }
}
// cluster leftover by centroid x
let leftoverClusters = clusterStrokeIndicesByX(leftoverIndices,
strokes: strokes)

// build final segments
var segments: [Segment] = []

for (obs, idxs) in boxToStrokes {
    var pts: [CGPoint] = []
    for i in idxs { pts.append(contentsOf: strokes[i]) }
    let bbox = VNImageRectForNormalizedRect(obs.boundingBox,
Int(imageSize.width), Int(imageSize.height))

```

```

        let recognized = obs.topCandidates(1).first?.string
        segments.append(Segment(points: pts, boundingBox: bbox,
recognizedString: recognized))
    }

    for cluster in leftoverClusters {
        var pts: [CGPoint] = []
        for i in cluster { pts.append(contentsOf: strokes[i]) }
        // derive bbox from points
        let minX = pts.map { $0.x }.min() ?? 0
        let minY = pts.map { $0.y }.min() ?? 0
        let maxX = pts.map { $0.x }.max() ?? 0
        let maxY = pts.map { $0.y }.max() ?? 0
        let bbox = CGRect(x: minX, y: minY, width: maxX-minX, height: maxY-
minY)

        segments.append(Segment(points: pts, boundingBox: bbox,
recognizedString: nil))
    }

    // sort segments left-to-right for stability
    segments.sort { ($0.boundingBox?.minX ?? 0) < ($1.boundingBox?.minX ??
0) }

    return segments
}

private static func clusterStrokeIndicesByX(_ indices: [Int], strokes:
[[CGPoint]]) -> [[Int]] {
    // simple single-linkage clustering by centroid x distance
    let centroids = indices.map { i -> CGFloat in
        let s = strokes[i]
        guard !s.isEmpty else { return 0 }
        return s.map { $0.x }.reduce(0, +) / CGFloat(s.count)
    }
    let zipped = zip(indices, centroids).sorted(by: { $0.1 < $1.1 })
    var clusters: [[Int]] = []
    var current: [Int] = []
    var lastX: CGFloat? = nil
    for (idx, x) in zipped {
        if let lx = lastX, abs(x - lx) > 40 { // threshold relative to
canvas (adjust)
            if !current.isEmpty { clusters.append(current) }

```

```

        current = [idx]
    } else {
        current.append(idx)
    }
    lastX = x
}
if !current.isEmpty { clusters.append(current) }
return clusters
}
}

// MARK: - Feature extractor

fileprivate struct FeatureExtractor {
    /// Slant in degrees (fit line to points)
    static func slantAngleDegrees(points: [CGPoint]) -> Double {
        guard points.count > 2 else { return 0 }
        let xs = points.map { Double($0.x) }
        let ys = points.map { Double($0.y) }
        let xMean = xs.reduce(0, +) / Double(xs.count)
        let yMean = ys.reduce(0, +) / Double(ys.count)
        var num = 0.0, den = 0.0
        for (x, y) in zip(xs, ys) {
            num += (x - xMean) * (y - yMean)
            den += (x - xMean) * (x - xMean)
        }
        if den == 0 { return 0.0 }
        let slope = num / den // dy/dx
        let angleRad = atan(slope)
        let angleDeg = angleRad * 180.0 / .pi
        // convert to signed slant relative to vertical if you prefer; keep
horizontal-based here
        return angleDeg
    }

    /// Curvature discrete approximation along polyline
    static func curvatures(points: [CGPoint]) -> [Double] {
        guard points.count >= 3 else { return [] }
        var k: [Double] = []
        for i in 1..

```

```

        let dx1 = Double(p1.x - p0.x), dy1 = Double(p1.y - p0.y)
        let dx2 = Double(p2.x - p1.x), dy2 = Double(p2.y - p1.y)
        let cross = abs(dx1 * dy2 - dy1 * dx2)
        let a = sqrt(dx1*dx1 + dy1*dy1)
        // stable small denom guard
        let denom = max(1e-6, pow((dx1*dx1 + dy1*dy1), 1.5) + 1e-6)
        let curvature = cross / denom
        k.append(curvature)
    }
    return k
}

/// Returns relative proportions (xHeightRatio etc.) after normalization
static func proportions(points: [CGPoint], baselineInfo: BaselineInfo,
drawingSize: CGSize) -> (xHeightRatio: Double, ascenderRatio: Double,
descenderRatio: Double) {
    guard !points.isEmpty else { return (0.0, 0.0, 0.0) }
    let minY = points.map { $0.y }.min() ?? 0
    let maxY = points.map { $0.y }.max() ?? 0
    let totalH = maxY - minY
    let xh = baselineInfo.xHeight
    let xHeightRatio = totalH > 0 ? Double(xh / totalH) : 0.0
    // asc/desc relative to xHeight
    let asc = max(0.0, Double((baselineInfo.ascenderLine -
baselineInfo.baselineY) / xh))
    let desc = max(0.0, Double((baselineInfo.descenderLine -
baselineInfo.baselineY) / xh))
    return (xHeightRatio, asc, desc)
}
}

// MARK: - DTW (Dynamic Time Warping) for shape similarity

fileprivate struct DTW {
    // sequences are arrays of normalized CGPoints (0..1)
    static func distance(sequenceA: [CGPoint], sequenceB: [CGPoint]) -> Double
{
    let n = sequenceA.count
    let m = sequenceB.count
    guard n > 0 && m > 0 else { return Double.infinity }
    // compute cost matrix with O(n*m)

```

```

    var dtw = Array(repeating: Array(repeating: Double.infinity, count:
m+1), count: n+1)
    dtw[0][0] = 0.0
    for i in 1...n {
        for j in 1...m {
            let a = sequenceA[i-1], b = sequenceB[j-1]
            let d = hypot(Double(a.x - b.x), Double(a.y - b.y))
            let cost = d
            let minPrev = min(dtw[i-1][j], dtw[i][j-1], dtw[i-1][j-1])
            dtw[i][j] = cost + minPrev
        }
    }
    return dtw[n][m] / Double(n + m) // normalized by path length
}

```

// MARK: - Scoring helpers

```

fileprivate struct Scoring {
    struct Weights {
        let legibility: Double = 0.30
        let shape: Double = 0.25
        let slant: Double = 0.15
        let proportion: Double = 0.10
        let connection: Double = 0.10
        let curvature: Double = 0.06
        let timing: Double = 0.04
    }

    static func slantScore(for slantDeg: Double, targetSlantDeg: Double,
toleranceDeg: Double) -> Double {
        let err = abs(slantDeg - targetSlantDeg)
        if err <= toleranceDeg { return 100.0 }
        // linear falloff
        let score = max(0.0, 100.0 - (err - toleranceDeg) * 5.0)
        return score
    }

    static func proportionScore(xHeightRatio: Double, ideal: Double, tolerance:
Double) -> Double {
        let err = abs(xHeightRatio - ideal)

```

```

        if err <= tolerance { return 100.0 }
        return max(0.0, 100.0 - (err - tolerance) * 100.0)
    }

    static func curvatureScore(meanCurvature: Double) -> Double {
        // smaller is smoother; choose mapping heuristically
        let score = max(0.0, 100.0 - meanCurvature * 500.0)
        return score
    }

    static func connectionScore(for seg: Segment, baselineInfo: BaselineInfo) -
> Double {
        // quick heuristic: check continuity at endpoints if multiple strokes
        exist in segment
        let pts = seg.points
        if pts.count < 4 { return 80.0 } // short letters, neutral
        let first = pts.first!, last = pts.last!
        // distance between end and start if this is a join with adjacent
        letter would be checked elsewhere;
        // for single-letter, penalize if start and end are far (broken)
        let dist = hypot(last.x - first.x, last.y - first.y)
        // scale relative to baseline xHeight
        let scale = baselineInfo.xHeight > 0 ? baselineInfo.xHeight : 10.0
        let norm = dist / scale
        return max(0.0, 100.0 - norm * 40.0)
    }

    static func timingScore(for seg: Segment) -> Double {
        // if stroke timestamps available we could compute speed variance; in
        prototype we assume neutral
        return 70.0
    }
}

// MARK: - Utility: Coordinate normalizer

fileprivate struct CoordinateNormalizer {
    /// Normalize points to 0..1 space using bounding box of points
    static func normalize(points: [CGPoint], in imageRect: CGRect) -> [CGPoint]
    {
        guard !points.isEmpty else { return points }

```

```

    let minX = points.map { $0.x }.min() ?? 0
    let maxX = points.map { $0.x }.max() ?? 1
    let minY = points.map { $0.y }.min() ?? 0
    let maxY = points.map { $0.y }.max() ?? 1
    let width = max(1, maxX - minX)
    let height = max(1, maxY - minY)
    return points.map { p in
        let nx = (p.x - minX) / width
        // normalize y so origin at top-> keep same orientation
        let ny = (p.y - minY) / height
        return CGPoint(x: nx, y: ny)
    }
}

}

// MARK: - String similarity (Levenshtein)

fileprivate struct StringSimilarity {
    static func levenshteinDistance(_ a: String, _ b: String) -> Int {
        let s = Array(a), t = Array(b)
        let n = s.count, m = t.count
        if n == 0 { return m }
        if m == 0 { return n }
        var v0 = [Int](repeating: 0, count: m+1)
        var v1 = [Int](repeating: 0, count: m+1)
        for j in 0...m { v0[j] = j }
        for i in 0..

```

```

        return 1.0 - (Double(d) / Double(maxLen))
    }
}

// MARK: - Vision helpers (text recognition + contours fallback)

fileprivate struct VisionHelpers {
    static func recognizeTextObservations(in image: UIImage) throws ->
[VNRecognizedTextObservation] {
        guard let cg = image.cgImage else { return [] }
        let req = VNRecognizeTextRequest()
        req.recognitionLanguages = ["en-US"]
        req.recognitionLevel = .accurate
        req.usesLanguageCorrection = true
        let handler = VNImageRequestHandler(cgImage: cg, options: [:])
        try handler.perform([req])
        return req.results as? [VNRecognizedTextObservation] ?? []
    }

    static func extractContours(from cgImage: CGImage) throws -> [[CGPoint]] {
        // Use VNDetectContoursRequest to extract stroke-like contours if
        stroke sampling unavailable
        let request = VNDetectContoursRequest()
        request.contrastAdjustment = 1.0
        request.maximumImageDimension = 1024
        let handler = VNImageRequestHandler(cgImage: cgImage, options: [:])
        try handler.perform([request])
        guard let observation = request.results?.first as?
VNContoursObservation else { return [] }
        var result: [[CGPoint]] = []
        for i in 0..

```



```
// MARK: - Convenience: PKDrawing -> UIImage
```

```
fileprivate extension PKDrawing {  
    func asImage(background-color: UIColor = .white, scale: CGFloat = 1.0) ->  
    UIImage {  
        // determine bounds  
        let bounds = self.bounds  
        let width = max(1, bounds.width)  
        let height = max(1, bounds.height)  
        let size = CGSize(width: width * scale, height: height * scale)  
  
        let format = UIGraphicsImageRendererFormat()  
        format.scale = 1.0 // we handle scale manually  
        let renderer = UIGraphicsImageRenderer(size: size, format: format)  
        let img = renderer.image { ctx in  
            backgroundColor.setFill()  
            ctx.fill(CGRect(origin: .zero, size: size))  
            ctx.cgContext.scaleBy(x: scale, y: scale)  
            ctx.cgContext.translateBy(x: -bounds.minX, y: -bounds.minY)  
            self.drawHierarchy(in: bounds, with: .clear)  
        }  
        return img  
    }  
}
```

```
// MARK: - Small helpers
```

```
fileprivate extension Array where Element == Double {  
    func mean() -> Double {  
        guard !self.isEmpty else { return 0.0 }  
        return self.reduce(0.0, +) / Double(self.count)  
    }  
}
```

If you want next:

- I can convert this into a ready-to-run Xcode Playground / sample app showing the full UI (PencilKit canvas, “Analyze” button, JSON viewer).
- Or I can produce unit tests for ``DTW``, ``curvatures``, and ``slantAngleDegrees`` to validate numeric stability.
- I can also generate a set of canonical templates (SVG or simple normalized point arrays) for standard English cursive lowercase letters to use with the DTW comparator.

Which should I do next? (I recommend the sample app so you can physically test with an Apple Pencil on iPad.)



definitely the sample app...



Perfect — I’ll lay out a **self-contained Swift prototype** for your cursive handwriting tutor, with the full stroke capture → recognition → scoring pipeline, but simplified enough to run on an iPad right away.

This prototype will:

1. Use **PencilKit** to capture strokes.
2. Use **Vision** to get recognized text.
3. Run a **basic stroke analysis** (slant, baseline deviation).
4. Compare against the target phrase.
5. Display **feedback** in the UI.

SwiftUI + PencilKit + Vision Prototype

```
import SwiftUI
import PencilKit
import Vision

struct ContentView: View {
    @State private var canvasView = PKCanvasView()
    @State private var recognizedText: String = ""
    @State private var feedback: String = ""

    // Target phrase for the lesson
    private let targetPhrase = "loop"

    var body: some View {
        VStack {
            Text("Write in cursive: \(targetPhrase)")
                .font(.title)
                .padding()

            WritingCanvas(canvasView: $canvasView)
                .frame(height: 300)
                .border(Color.gray, width: 1)
                .padding()

            HStack {
                Button("Clear") {
                    canvasView.drawing = PKDrawing()
                    recognizedText = ""
                    feedback = ""
                }

                Spacer()

                Button("Evaluate") {
                    evaluateDrawing()
                }
            }
        }
        .padding()
    }
}
```

```

        Text("Recognized: \$(recognizedText)")
            .font(.headline)
            .padding(.top)

        Text("Feedback: \$(feedback)")
            .foregroundColor(.blue)
            .padding(.top)

        Spacer()
    }
    .padding()
}

// Convert PKDrawing to UIImage
private func drawingToImage() -> UIImage {
    let imageBounds = canvasView.drawing.bounds
    return canvasView.drawing.image(from: imageBounds, scale: 2.0)
}

// Main evaluation pipeline
private func evaluateDrawing() {
    let image = drawingToImage()

    recognizeHandwriting(from: image) { recognized in
        DispatchQueue.main.async {
            self.recognizedText = recognized

            let textScore = compareText(targetPhrase, recognized)
            let strokeMetrics = analyzeStrokes(canvasView.drawing)

            self.feedback = """
            Text match: \$(Int(textScore * 100))%
            Avg slant: \$(String(format: "%.1f", strokeMetrics.slantAngle))°
            Baseline error: \$(String(format: "%.1f",
strokeMetrics.baselineError)) pts
            """
        }
    }
}

// Vision text recognition

```

```

private fun recognizeHandwriting(from image: UIImage, completion:
@escaping (String) -> Void) {
    let request = VNRecognizeTextRequest { request, _ in
        let recognized = request.results?
            .compactMap { ($0 as?
VNRecognizedTextObservation)?.topCandidates(1).first?.string }
            .joined(separator: " ") ?? ""
        completion(recognized)
    }
    request.recognitionLanguages = ["en-US"]
    request.recognitionLevel = .accurate
    request.usesLanguageCorrection = true

    guard let cgImage = image.cgImage else { return }
    let handler = VNImageRequestHandler(cgImage: cgImage)
    try? handler.perform([request])
}

// Simple text similarity score
private fun compareText(_ target: String, _ student: String) -> Double {
    let distance = levenshtein(target.lowercased(), student.lowercased())
    let maxLen = max(target.count, student.count)
    return maxLen > 0 ? 1.0 - (Double(distance) / Double(maxLen)) : 0
}

// Levenshtein distance
private fun levenshtein(_ s: String, _ t: String) -> Int {
    let m = Array(s)
    let n = Array(t)
    var dist = [[Int]](repeating: [Int](repeating: 0, count: n.count + 1),
count: m.count + 1)

    for i in 0...m.count { dist[i][0] = i }
    for j in 0...n.count { dist[0][j] = j }

    for i in 1...m.count {
        for j in 1...n.count {
            if m[i-1] == n[j-1] {
                dist[i][j] = dist[i-1][j-1]
            } else {
                dist[i][j] = min(dist[i-1][j-1], dist[i][j-1], dist[i-1][j])
            }
        }
    }
}

```

```

[j]) + 1
        }
    }
}
return dist[m.count][n.count]
}

// Stroke geometry analysis
private func analyzeStrokes(_ drawing: PKDrawing) -> (slantAngle: Double,
baselineError: Double) {
    var angles: [Double] = []
    var yOffsets: [CGFloat] = []

    for stroke in drawing.strokes {
        let points = stroke.path.flatMap { $0.points }
        for i in 1..

```

```
        canvasView.backgroundColor = .white
        canvasView.allowsFingerDrawing = false
        return canvasView
    }

    func updateUIView(_ uiView: PKCanvasView, context: Context) {}
}
```

What This Prototype Does

- Lets the user **write** the target phrase with Apple Pencil.
- Captures **stroke data** via PencilKit.
- Uses Vision to **recognize handwriting**.
- Runs a simple **geometry-based slant & baseline analysis**.
- Compares recognized text to the **lesson target**.
- Displays **accuracy, slant angle, baseline deviation**.

Next Steps After This

1. Replace geometric analysis with a **trained Core ML model** for better style scoring.
2. Add **per-letter overlays** showing exact trouble spots.
3. Introduce **animated stroke demonstrations** for each lesson.
4. Store results in **Core Data / CloudKit** for student progress tracking.

If you want, I can now extend this into a **multi-lesson app** with:

- A lesson selector
- Animated cursive examples
- Progress tracking
- Adaptive difficulty

That would make it a full-fledged teaching tool instead of a single-screen demo.

Do you want me to build that next?